

BÀI 7: GIAO TIẾP VỚI BỘ NHỚ

1. MỤC ĐÍCH

Thông qua bài thực hành này, sinh viên sẽ biết cách:

- Thực hiện các bước để mô phỏng toàn bộ hệ thống (phần cứng lẫn phần mềm) dùng công cụ Modelsim

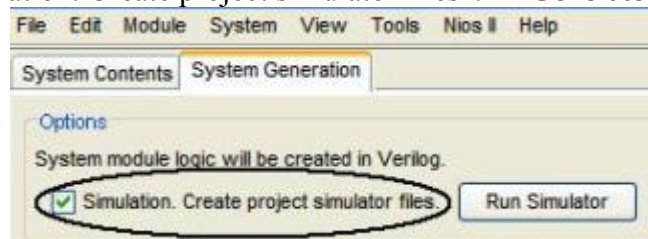
Tài liệu tham khảo:

- [1] Modelsim Advanced Verification and Debugging Altera Tutorial
- [2] Modelsim Tutorial

2. PHÂN LÝ THUYẾT

2.1 Xây dựng phần cứng

- Trong bài thực hành này, sinh viên sẽ mô phỏng hệ thống hoàn chỉnh của Bài 5 (CPU NIOS II/s, On-chip Memory, JTAG UART, bộ nhớ 4 thanh ghi tự thiết kế) dùng công cụ Modelsim.
- Trong công cụ Quartus II, tạo project lab7_part1 và thực hiện lại các bước tạo hệ thống trên SoPC Builder như lab5_part1 (Bài 5). Đặt tên hệ thống là nios_sys, và tên của On-chip Memory vẫn để mặc định là onchip_memory2_0 (đây là quy ước để dễ hướng dẫn về sau)
- Khi xây dựng xong hệ thống trên SoPC Builder, trước khi nhấn Generate, lưu ý phải check vào mục “Simulation. Create project simulator files”. → Generate.



Hình 2.1 – Check để tạo các files cần thiết cho mô phỏng phần cứng

- Trong thư mục project bây giờ sẽ có một thư mục con <tên hệ thống>_sim (trong trường hợp này là nios_sys_sim). Đây chính là thư mục sẽ chứa các files cần thiết để mô phỏng hệ thống.
- Trong công cụ Quartus II, tạo file top-level lab7_part1.v có nội dung giống lab5_part1.v → Compile project.

2.2 Xây dựng testbench để mô phỏng

- Tạo file lab7_part1_tb.v với nội dung như Hình 2.2. Add file này vào project lab7_part1.

```
1 module lab7_part1_tb();
2 reg clk;
3 reg reset_n;
4 parameter PERIOD = 10;
5 initial
6 begin
7     clk <= 1'b0;
8     forever #(PERIOD) clk = ~ clk;
9 end
10 initial
11 begin
12     reset_n = 1'b1;
13     #20 reset_n = 1'b0;
14     #60 reset_n = 1'b1;
```

```

15 end
16 lab7_part1 DUT (
17     clk,
18     reset_n
19 );
20 endmodule

```

Hình 2.2 – Nội dung file testbench lab7_part1_tb.v

2.3 Xây dựng phần mềm

- Tạo project lab7_part1 và viết code C đơn giản để ghi dữ liệu lên bộ nhớ 4 thanh ghi như sau

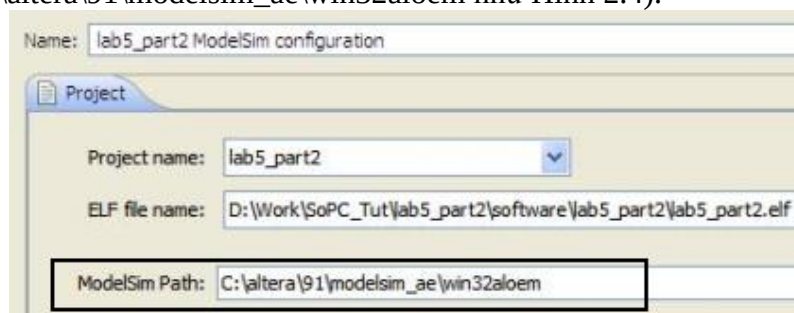
```

1 #include "system.h"
2 void main()
3 {
4     int * mem_ptr = (int *) MEMORY_0_BASE;
5     int i;
6     for(i = 0; i < 4; i++)
7         *(mem_ptr + i) = 2 * i;
8 }

```

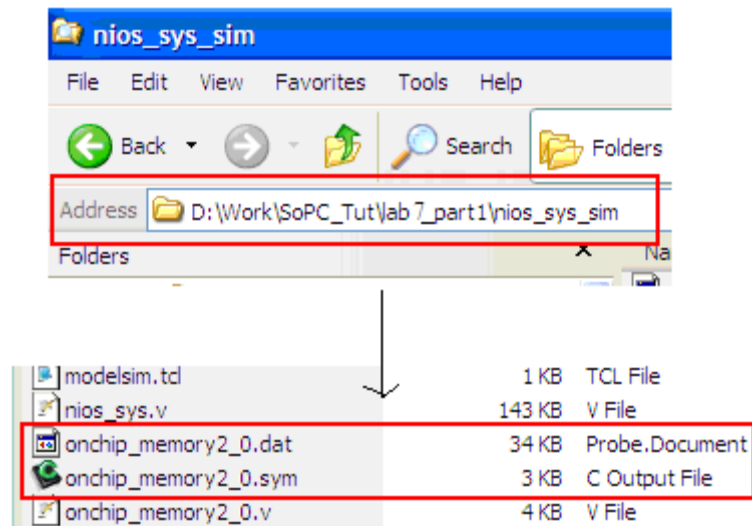
Hình 2.3 – Nội dung file lab7_part1.c

- Tiến hành build project → khi build xong, trong thư mục project Quartus sẽ xuất hiện file **.hex** (onchip_memory2_0.hex). Đây là file chứa nội dung của code C sau khi biên dịch xong.
- Để công cụ Nios II Eclipse Platform tạo ra các files dành cho mô phỏng, ta có 2 cách sau
 - ✓ **Cách 1:** Sử dụng giao diện
 - Trong công cụ Nios II Eclipse Platform → click phải lên project → Run As → 4 Nios Hardware.
 - Sau 1 lúc nếu có cửa sổ Run Configurations xuất hiện thì ta sẽ trở tới đường dẫn của chương trình Modelsim trong mục Modelsim Path (nếu cài đặt Modelsim 6.5b của bộ Quartus 9.1 thì mặc định nó sẽ nằm trong thư mục C:\altera\91\modelsim_ae\win32aloem như Hình 2.4).



Hình 2.4 – Mở Modelsim dùng mô phỏng

- Sau khi thiết lập Modelsim Path → chọn Apply → Run.
- Tiếp tục chờ cho tới khi phần mềm Modelsim được tự động mở lên. Sau đó vào thư mục ...**<tên project>\nios_sys_sim**, kiểm tra xem đã có các file **onchip_memory2_0.dat** và **onchip_memory2_0.sym** chưa (file onchip_memory2_0.dat phải có dung lượng khác 0KB). Đây là 2 files phục vụ cho việc mô phỏng.



Hình 2.5 – Các file .dat và .sym trong thư mục nios_sys_sim

- Lưu ý: nếu sau khi Run và Modelsim bắt đầu được mở mà ta thấy *Progress* trong công cụ Nios II Eclipse Platform vẫn dừng lại ở mức 57% thì có nghĩa là các files phục vụ mô phỏng (.sym) không được tạo ra và file .dat cũng chưa được cập nhật. Do đó không thể nào tiến hành mô phỏng được. Trong trường hợp xảy ra điều này thì ta phải sử dụng Cách 2.
- ✓ **Cách 2:** Sử dụng dòng lệnh
 - Trong công cụ Nios II Eclipse Platform, click phải lên project → Nios II → Nios II Command Shell..., rồi đợi tới khi có dấu nhắc **[Nios II EDS]\$** xuất hiện → gõ lệnh **make clean** và chờ thông báo clean complete như Hình 2.6.

```

Welcome To Altera SOPC Builder
Version 9.1, Built Wed Oct 21 22:39:30 PDT 2009

-----

Welcome to the Nios II Embedded Design Suite
Version 9.1, Built Thu Oct 22 02:02:11 PDT 2009

Example designs can be found in
/cygdrive/c/altera/91/nios2eds/examples

-----

<You may add a startup script: c:/altera/91/nios2eds
/cygdrive/d/Work/SoPC_Tut/lab5_part1/software/lab7
[NiosII EDS]$ make clean
[lab7_part1 clean complete]
  
```

Hình 2.6 – Cửa sổ Command Shell và gõ lệnh make clean

- Sau đó, gõ tiếp lệnh **make mem_init_install**, và chờ tới khi dấu nhắc xuất hiện trở lại như Hình 2.7. Lúc này các files dùng cho mô phỏng đã được tạo ra đầy đủ → đóng cửa sổ Command Shell.

```

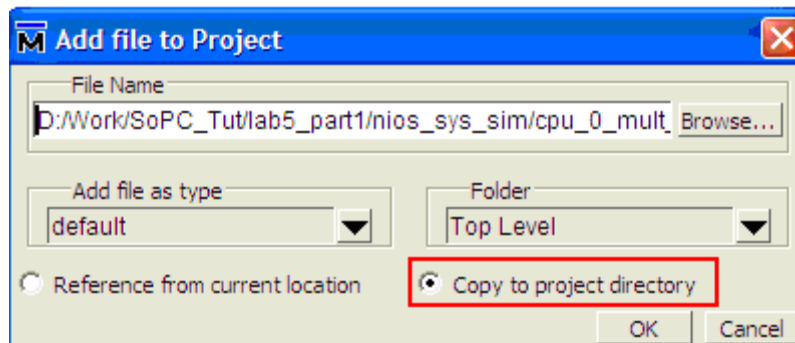
/cygdrive/d/Work/SoPC_Tut/lab5_part1/software/lab7_part1
[NiosII EDS] $ make mem_init_install
Info: Compiling lab7_part1.c to obj/lab7_part1.o
nios2-elf-gcc -xc -MP -MMD -c -I../lab7_part1_bsp -I../lab7_part1_bsp/drivers/in
c -I../lab7_part1_bsp/HAL/inc -DSYSTEM_BUS_WIDTH=32 -D__hal__ -pipe -DALT_NO_IN
STRUCTION_EMULATION -DALT_USE_SMALL_DRIVERS -DALT_SINGLE_THREADED -O0 -g -Wall
lab7_part1.c:10: warning: return type of 'main' is not 'int'
Info: Building ../lab7_part1_bsp
make --no-print-directory -C ../lab7_part1_bsp
[BSP build complete]
Info: Linking lab7_part1.elf
nios2-elf-g++ -T'../lab7_part1_bsp/linker.x' -msys-crt0='../lab7_part1_bsp/obj/
HAL/src/crt0.o' -msys-lib=hal_bsp -L../lab7_part1_bsp -msmallc -Wl,-Map=lab7_p
art1.map -O0 -g -Wall -mno-hw-div -mhw-mul -mno-hw-mulx -o lab7_part1.elf o
bj/lab7_part1.o -lm
nios2-elf-insert lab7_part1.elf --cpu_name cpu_0 --simulation_enabled true --std
err_dev jtag_uart_0 --stdin_dev jtag_uart_0 --stdout_dev jtag_uart_0 --sopc_syst
em_name nios_sys --quartus_project_dir "d:/Work/SoPC_Tut/lab5_part1" --jdi d:/Wo
rk/SoPC_Tut/lab5_part1/software/lab7_part1_bsp/../../lab5_part1.jdi
Info: (lab7_part1.elf) 9324 Bytes program size (code + initialized data).
Info: 22 KBytes free for stack + heap.
Post-processing to create mem_init/onchip_memory2_0.hex...
elf2hex lab7_part1.elf 0x00008000 0x0000ffff --width=32 \
--create-lanes=0 mem_init/onchip_memory2_0.hex
Post-processing to create mem_init/hdl_sim/onchip_memory2_0.dat...
elf2dat --infile=lab7_part1.elf --outfile=mem_init/hdl_sim/onchip_memory2_0.dat
\
--base=0x00008000 --end=0x0000ffff --width=32 \
--create-lanes=0
Post-processing to create mem_init/hdl_sim/onchip_memory2_0.sym...
nios2-elf-nm -n lab7_part1.elf > mem_init/hdl_sim/onchip_memory2_0.sym
'mem_init/onchip_memory2_0.hex' -> '...../onchip_memory2_0.hex'
'mem_init/hdl_sim/onchip_memory2_0.dat' -> '...../nios_sys_sim/onchip_memory2_0
.dat'
'mem_init/hdl_sim/onchip_memory2_0.sym' -> '...../nios_sys_sim/onchip_memory2_0
.sym'
/cygdrive/d/Work/SoPC_Tut/lab5_part1/software/lab7_part1
[NiosII EDS] $

```

Hình 2.7 – Hoàn tất việc tạo files mô phỏng

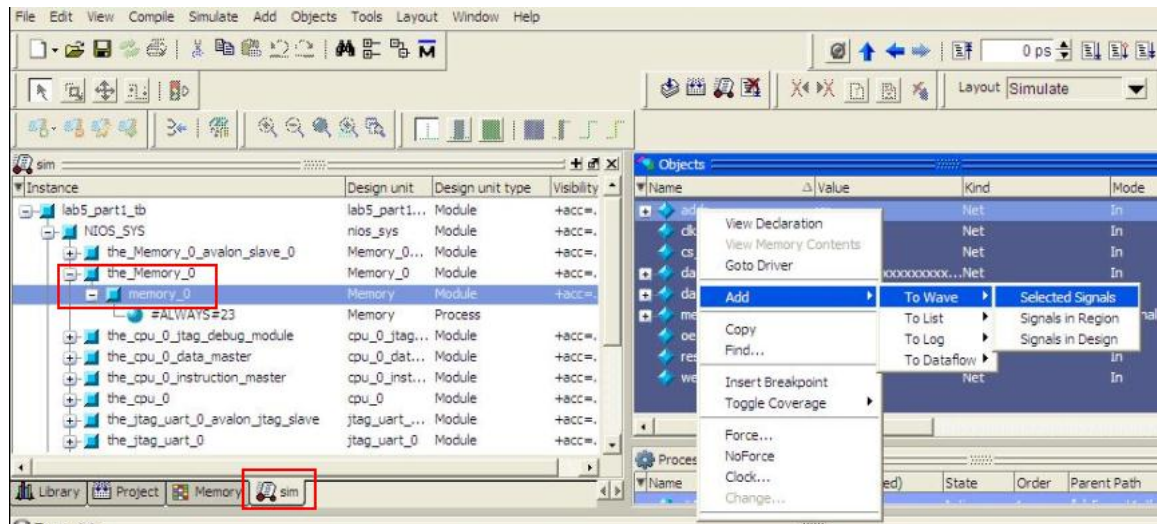
2.4 Mô phỏng dùng Modelsim

- Nếu sử dụng Cách 1 (dùng giao diện) Modelsim được tự động mở lên có thể xuất hiện lỗi *# ** Error: (vsim-19) Failed to access library 'nios_sys_sim' at "nios_sys_sim" → bỏ qua lỗi này.*
- Nếu sử dụng Cách 2 (dùng lệnh) thì ta sẽ tự mở Modelsim bằng tay nên sẽ không gặp lỗi trên → Tạo project mới bằng cách chọn File → New → Project... → đặt tên project là lab7_part1 nằm trong thư mục "...\\<thư mục project Quartus>\\nios_sys_sim" → add existing file → chọn add tất cả các file Verilog trong thư mục project Quartus (trừ 2 file cpu.v và nios_sys_inst.v) và file **cpu.vo** → Check vào ô "Copy to project directory" → chọn OK.



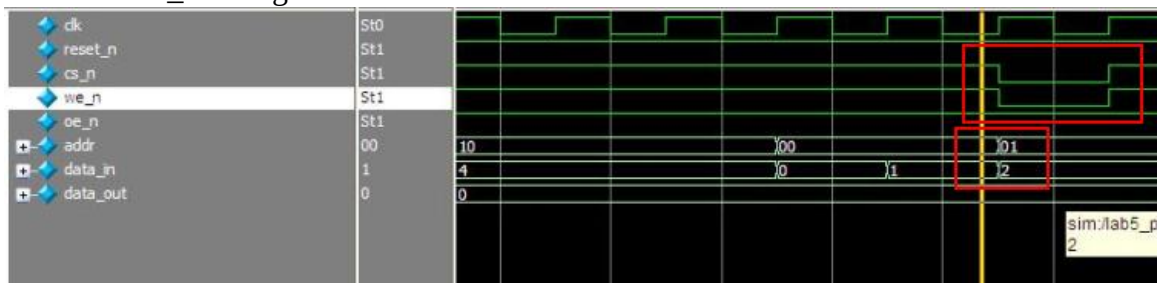
Hình 2.8 – Add files vào project

- Trong phần cửa sổ Transcript, gõ lệnh **vsim -novopt lab7_part1_th**
- Trong cửa sổ mô phỏng → chọn tab sim, chọn instance memory 4 thanh ghi (Hình 2.9) → chuyển qua phần cửa sổ Objects → phải chuột lên một tín hiệu bất kỳ → Add → To Wave → Signals in Region để add hết các tín hiệu của module bộ nhớ 4 thanh ghi.



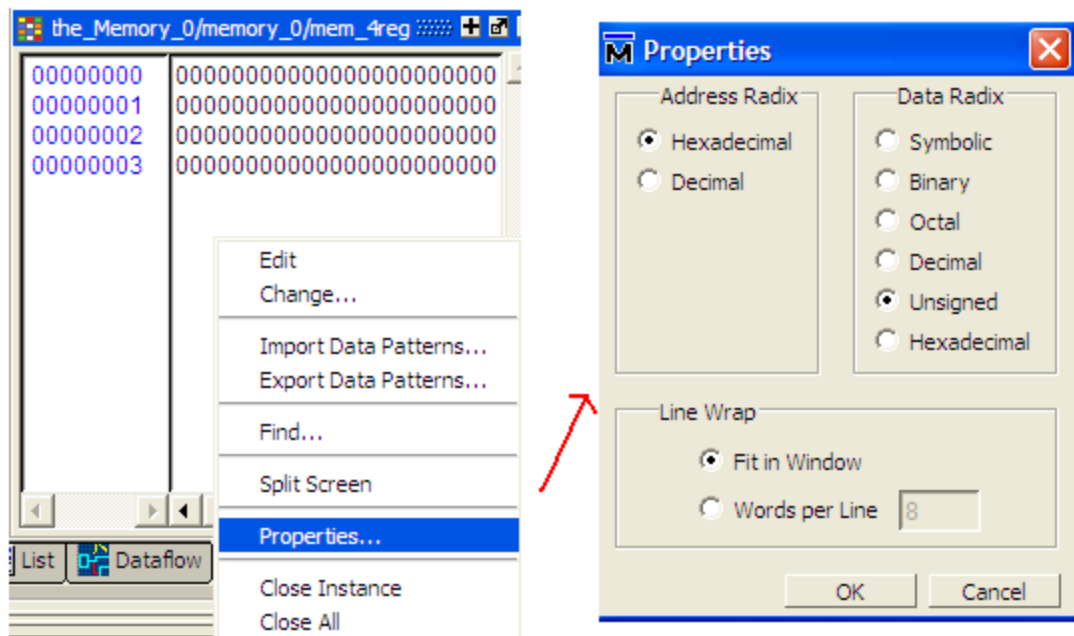
Hình 2.9 – Thêm các tín hiệu để quan sát

- Sau khi add tín hiệu, cửa sổ Wave sẽ hiện ra (hoặc ta gõ lệnh **view wave** trong Transcript)
- Chọn thời gian chạy mô phỏng là 500ns bằng cách chỉnh tại
- Click vào (bên trái khung chỉnh thời gian) để restart
- Chạy mô phỏng bằng cách click vào biểu tượng (bên phải khung chỉnh thời gian)
- Quan sát kết quả
 - ✓ **Quan sát dạng sóng**
 - Để dễ xem kết quả, ta chọn kiểu dữ liệu hiển thị cho tín hiệu data_in và data_out là dạng unsigned bằng cách phải chuột lên tín hiệu → Radix → Hexadecimal
 - Khi đã mô phỏng hết thời gian yêu cầu 500 ns, ta tiến hành quan sát dạng sóng các tín hiệu:
 - Click vào để phóng to hay thu nhỏ dạng sóng
 - Trong bài này, dữ liệu sẽ được ghi vào bộ nhớ 4 thanh ghi khi chip select xuống 0. Vì vậy, ta sẽ tới chỗ cs_n có cạnh xuống để quan sát bằng cách chọn tín hiệu cs_n, rồi click vào hoặc (Find Next Transition) cho tới khi thấy chỗ tín hiệu cs_n xuống 0 như Hình 2.10.

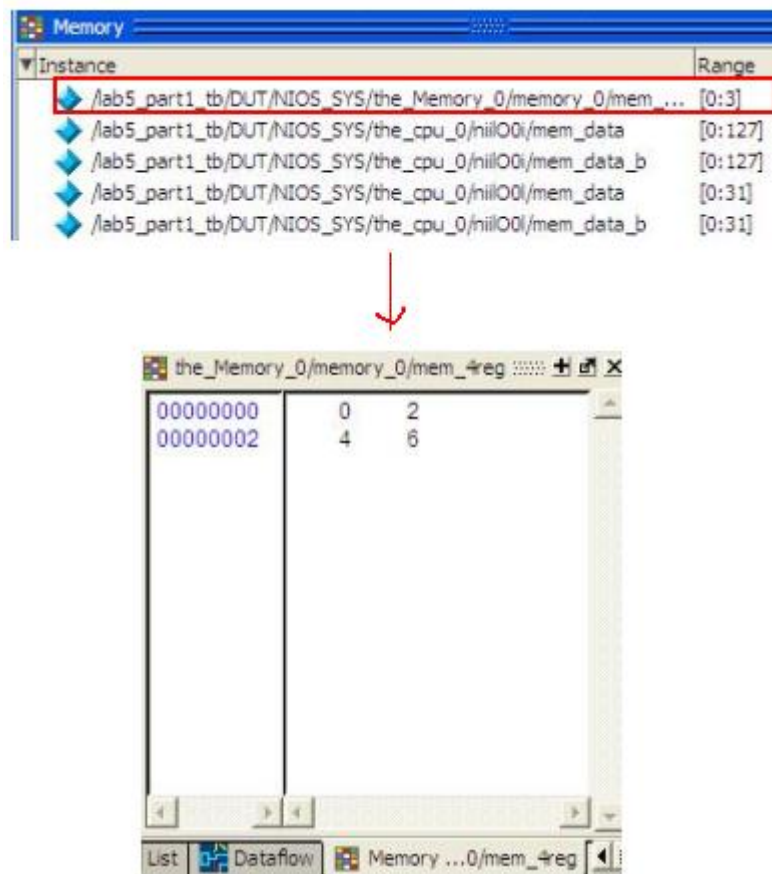


Hình 2.10 – Dạng sóng của các tín hiệu khi cs_n và we_n được tích cực

- ✓ **Quan sát bộ nhớ**
 - Sau khi chạy mô phỏng, dữ liệu đã được ghi vào bộ nhớ 4 thanh ghi giống như trong code C mô tả. Để quan sát 1 cách trực quan bộ nhớ, chọn View → Memory List (w).
 - Chọn tab memory → tìm và double click vào bộ nhớ có Range là [0:3]
 - Một phần cửa sổ mới xuất hiện có chứa dữ liệu của bộ nhớ ta vừa chọn (Hình 2.12). Để hiển thị dữ liệu dạng thập phân, click phải lên không gian của phần cửa sổ này → Properties → chọn Unsigned dưới mục Data Radix (Hình 2.11).



Hình 2.11 – Thay đổi Data Radix



Hình 2.12 – Dữ liệu đã được ghi vào bộ nhớ 4 thanh ghi

BÁO CÁO THỰC HÀNH

BÀI 7: MÔ PHỎNG HỆ THỐNG SoPC DÙNG MODELSIM

Sinh viên:

Lớp: Nhóm:

Phần chuẩn bị trước ở nhà

*** Sinh viên nộp cho giảng viên trước khi vào thực hành**

Bài 1: Sau khi quan sát dạng sóng, để lưu lại format của cửa sổ Wave (để lần sau mô phỏng không cần phải add tín hiệu lại vào Wave nữa), thì thực hiện như thế nào?

.....

.....

.....

.....

.....

.....

.....

.....

Bài 2: Nêu vắn tắt lại các bước để xây dựng một hệ thống (phần cứng + phần mềm) và mô phỏng nó dùng Modelsim.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

BÁO CÁO THỰC HÀNH

BÀI 7: MÔ PHỎNG HỆ THỐNG SoPC DÙNG MODELSIM

Sinh viên:

Lớp: Nhóm:

Phần thực hành trên lớp

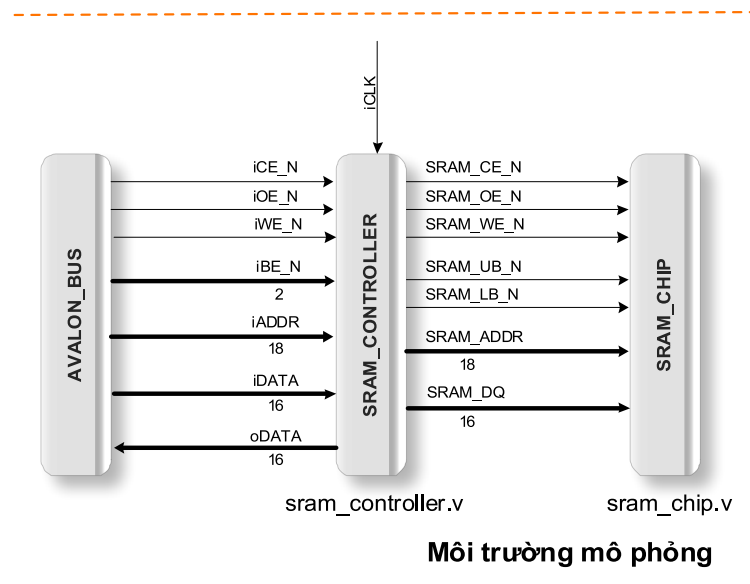
* Sinh viên nộp cho giảng viên sau buổi thực hành

Bài 1. Tạo project mới tên là lab7_part2. Xây dựng hệ thống với SRAM Controller như Bài 5.

- Hệ thống gồm: CPU Nios II/s, Memory Onchip 32 KB, JTAG UART, SRAM Controller.
- Viết code C trên Nios II Eclipse để ghi dữ liệu vào SRAM (dữ liệu ghi tùy ý). Tiến hành mô phỏng và quan sát dạng sóng các tín hiệu trên Modelsim (các tín hiệu cần quan sát giống như Bài 5 khi sử dụng SignalTap).

* Gợi ý:

- Để mô phỏng được hệ thống dùng SRAM Controller, ta cần xây dựng file mô phỏng SRAM chip và kết nối nó vào hệ thống SoPC.



- Tạo file top-level dùng trong mô phỏng. File này kết nối module top-level trong Quartus II với module sram_chip với nhau.

```

1 module lab7_part2_sim (
2     // Inputs
3     CLOCK_50,
4     KEY,
5 );
6
7 // Inputs
8 input          CLOCK_50;
9 input [0:0]    KEY;
10 // wires
11 wire [15:0]    SRAM_DQ;
12 wire [17:0]    SRAM_ADDR;
13 wire          SRAM_CE_N;
14 wire          SRAM_WE_N;
```

```

15 wire          SRAM_OE_N;
16 wire          SRAM_UB_N;
17 wire          SRAM_LB_N;
18
19 lab7_part2 LAB7_PART2 (
20     // Inputs
21     .clk          (CLOCK_50),
22     .reset_n      (KEY[0]),
23     // Bidirectionals
24     .SRAM_DQ      (SRAM_DQ),
25     // wires
26     .SRAM_ADDR    (SRAM_ADDR),
27     .SRAM_CE_N    (SRAM_CE_N),
28     .SRAM_WE_N    (SRAM_WE_N),
29     .SRAM_OE_N    (SRAM_OE_N),
30     .SRAM_UB_N    (SRAM_UB_N),
31     .SRAM_LB_N    (SRAM_LB_N)
32 );
33
34 sram_chip SRAM_CHIP (
35     .clk          (CLOCK_50),
36     .address      (SRAM_ADDR),
37     // Bidirectionals
38     .data         (SRAM_DQ),
39     // Inputs
40     .cs           (SRAM_CE_N),
41     .we           (SRAM_WE_N),
42     .oe           (SRAM_OE_N),
43     .ub           (SRAM_UB_N),
44     .lb           (SRAM_LB_N)
45 );
46
47 endmodule

```